H

Hope, Inc. ERP

A beginner's guide to building a database-driven business system, explained with its own screens

Student explainer · ERD · Annotated screenshots · Roles & rights · Build roadmap

October 2026

Contents

1 The big picture 2

1.1 Six modules . . . 2
1.2 Three roles . . . 2
1.3 How a web system works . . . 2

2 The database 3

2.1 The ERD . . . 3
2.2 Table dictionary . . . 3
2.3 Two columns added to every table: Status and Stamp . . . 4
2.4 User-management tables (proposed) . . . 5
2.5 Following one sale through the tables . . . 5

3 The screens, one module at a time 6

3.1 Getting in: landing, register, login, forgot password . . . 6
3.2 Dashboards: same system, different menus . . . 7
3.3 Customers: your first CRUD screen . . . 8
3.4 Products and their price history . . . 8
3.5 Human Resources: employees, jobs and departments . . . 9
3.6 Sales: a header and its detail lines . . . 11
3.7 Reports . . . 13
3.8 User management: accounts, blocking and rights . . . 15

4 Building it 17

4.1 Suggested build order . . . 17
4.2 From prototype to real system . . . 17
4.3 Test every role . . . 17
4.4 Key terms . . . 18

1 The big picture

Hope, Inc. ERP is a small business system that keeps customers, products, employees and sales in one place. “ERP” stands for *Enterprise Resource Planning*: instead of separate spreadsheets for each department, every module reads and writes the **same database**. This guide walks you through what the system does (the screens), how its data is organised (the ERD), and the order in which a first-time developer could build it.

How to read it: go in order — big picture → database → screens → build roadmap. Every screenshot is labelled with the role looking at it. The screens are small, so zoom in on the PDF to see details. Boxes mark a new **concept**, a common **watch out**, or something to **try** yourself.

1.1 Six modules

Module	What it manages	Main tables
Customers	Who we sell to: name, address, payment terms (Net 30, COD …)	CUSTOMER
Products	The catalogue and each product’s price over time	PRODUCT, PRICEHIST
Human Resources	Employees, job titles, departments and each employee’s job/salary history	EMPLOYEE, JOB, DEPARTMENT, JOBHISTORY
Sales	A sale (header) and the products on it (detail lines)	SALES, SALESDETAIL (PAYMENT in the ERD)
Reports	Read-only summaries you can print or save as PDF	Reads all of the above
User Management	Who may log in, their role, and what they may do	USERS, RIGHTS, USERRIGHTS (new, §2.4)

1.2 Three roles

The same screen looks different depending on who is signed in. A new account always starts as a **User**; only an existing administrator can promote it (Figure 3).

Role	Who	What they can do	What they see
Superadmin	System owner	Everything, including assigning any role and System Settings	All rows incl. deleted, plus Status and Stamp columns
Admin	Manager	Manage most modules and user rights, but cannot see or change Superadmin accounts	All rows incl. deleted, plus Status and Stamp columns
User	Staff	Only the modules and actions an Admin has granted (“rights”)	Active rows only; no Status or Stamp columns

1.3 How a web system works



Every screen in this guide is simply a **view of rows from a table**. The browser asks, the server checks who you are and what you may do, then runs SQL against the database and sends the result back as a page.

CONCEPT **CRUD** — nearly every screen is built from four operations: **C**reate (“+ Add …” button), **R**ead (the list and search box), **U**pdate (Edit) and **D**elete (Delete). Learn to build one CRUD screen well and you can build all of them.

2 The database

2.1 The ERD

An **Entity-Relationship Diagram** is the blueprint of the database. Each box is a **table**, the underlined name(s) are the **primary key** (what makes a row unique), and a line is a **relationship**. At the end of a line, a single bar means “one”, a three-pronged “crow’s foot” means “many”, and a small circle means “optional” (zero allowed).

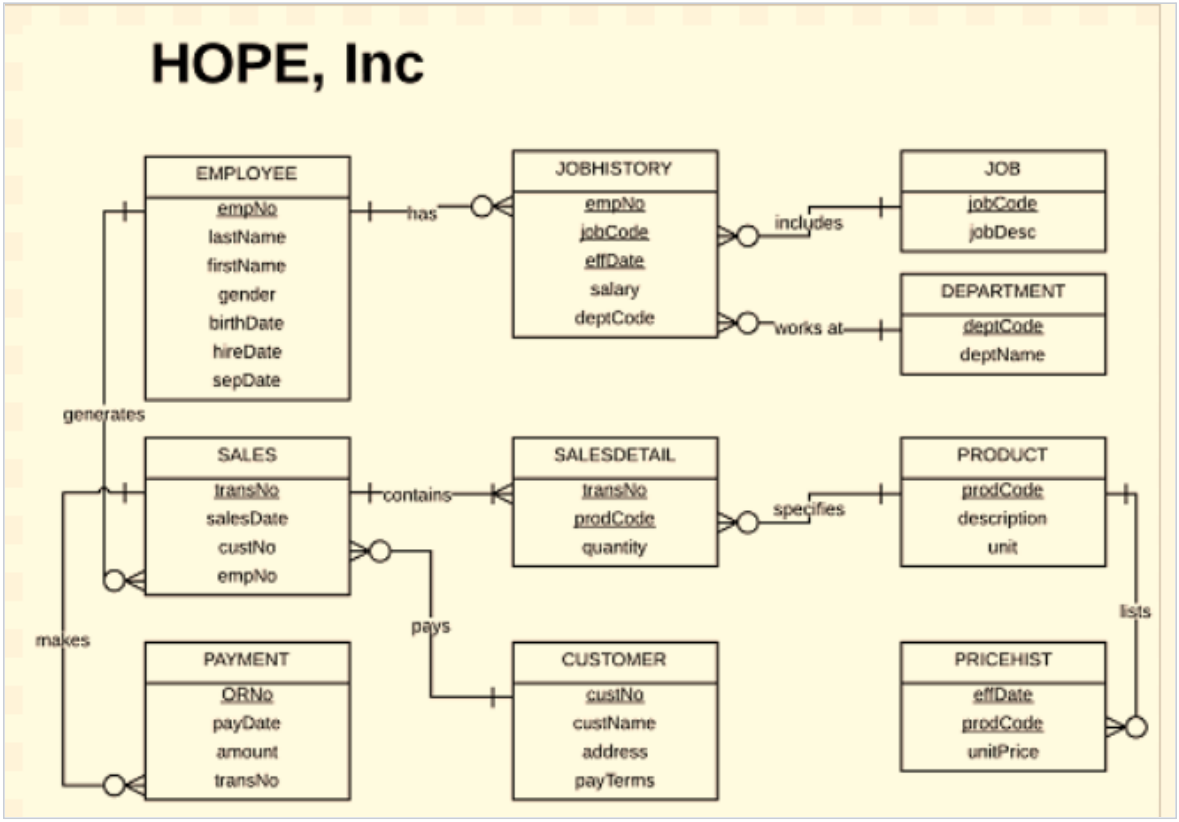


Figure 1. *ERD of the Hope, Inc. database. The Status and Stamp columns and the user-management tables are not drawn here; they are added in §2.3 and §2.4.*

2.2 Table dictionary

Table	One row is …	Primary key	Foreign keys (point to)
EMPLOYEE	a person who works or worked for Hope	empNo	—
JOB	a job title, e.g. Software Engineer	jobCode	—
DEPARTMENT	an organisational unit, e.g. Engineering	deptCode	—
JOBHISTORY	one job assignment of an employee, starting on a date, with a salary	empNo + jobCode + effDate	empNo → EMPLOYEE; jobCode → JOB; deptCode → DEPARTMENT
CUSTOMER	a buyer, with address and payment terms	custNo	—
PRODUCT	an item for sale (code, description, unit)	prodCode	—
PRICEHIST	the price of a product starting on a date	prodCode + effDate	prodCode → PRODUCT

Table	One row is …	Primary key	Foreign keys (point to)
SALES	the header of one sale (date, who bought, who sold)	transNo	custNo → CUSTOMER; empNo → EMPLOYEE
SALESDETAIL	one product line on a sale	transNo + prodCode	transNo → SALES; prodCode → PRODUCT
PAYMENT	one payment (official receipt) made on a sale	ORNo	transNo → SALES

Relationships in plain English

- One employee has many **job history** rows (every promotion, transfer or raise adds one). A job or department can appear in many history rows.
- One customer has many sales; one employee generates many sales; one sale contains many lines and may receive many payments.
- A sale can have many products and a product can be on many sales — a **many-to-many** relationship. **SALESDETAIL** sits in the middle as a *junction table* that turns it into two one-to-many links.
- One product has many prices over time, kept in **PRICEHIST**.

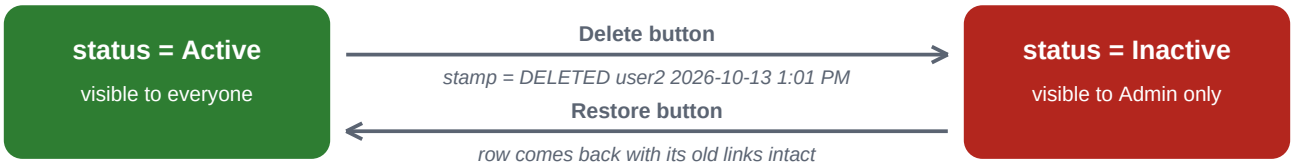
CONCEPT	**Primary key vs foreign key.** A primary key (PK) identifies a row inside its own table. A foreign key (FK) is a column that holds another table's PK, so the database can refuse a sale for a customer that does not exist. **Composite keys** use two or more columns together, as in JOBHISTORY (the same employee may hold the same job code again on a different date).

CONCEPT	**Derived data is not stored.** Notice SALES has no "total" column and PRODUCT has no "current price". Totals and current prices are *calculated* from SALESDETAIL and PRICEHIST each time, so they can never disagree with the source rows.

2.3 Two columns added to every table: Status and Stamp

The screens show a **Status** badge and a **Stamp** on every list. They were left out of the ERD to keep it readable, but each table (CUSTOMER, PRODUCT, PRICEHIST, EMPLOYEE, JOB, DEPARTMENT, JOBHISTORY, SALES, SALESDETAIL, and PAYMENT when you add its screen) needs both.

Column	Example value (from the screens)	Purpose
status	Active / Inactive	Marks whether the row is "alive". Deleting sets it to Inactive instead of removing the row (*soft delete*).
stamp	CREATED system 2026-10-01 9:00 AM EDITED user1 2026-10-13 1:01 PM DELETED user2 2026-10-13 1:01 PM	Audit trail of the *latest* action: what happened, who did it, and when.



Why not just delete the row? If you permanently deleted customer C-004, every sale that points to it would break. Soft delete keeps the history and links intact, and lets an Admin **Restore** a mistake. Admins and Superadmins see every row; a User sees only `status = 'Active'` rows and the Status and Stamp columns are hidden (compare Figures 9 and 10).

WATCH OUT

Two different "inactive" ideas. *Status* says the **record** was deleted from the system. *sepDate* (separation date) says the **employee** left the company. In the HR screens employee E-004 García has both: a separation date of 2025-12-31 and a deleted record. Do not mix them up.

The stamp keeps only the latest action. If you need the full history of every change, add a separate audit-log table later.

```
CREATE TABLE CUSTOMER (
  custNo    VARCHAR(10)  PRIMARY KEY,          -- 'C-001'
  custName  VARCHAR(100) NOT NULL,
  address   VARCHAR(200),
  payTerms  VARCHAR(20),                       -- 'Net 30', 'COD', 'Prepaid'
  status    VARCHAR(8)   NOT NULL DEFAULT 'Active',
  stamp     VARCHAR(80)                        -- 'CREATED system 2026-10-01 9:00 AM'
);
```

2.4 User-management tables (proposed)

Accounts, roles and rights are not in the ERD either. Here is a simple design that matches the User Management screens. Treat it as a starting point and adjust it to your own design.

Table	Key columns	Notes
USERS	userId (PK), firstName, lastName, email (unique), passwordHash, role, isBlocked, status, stamp	role is Superadmin, Admin or User. New registrations get User. Store a **hash** of the password, never the password itself.
RIGHTS	rightCode (PK), module, description	One row per checkbox in the Rights dialog: 36 rights in total (table in §3.8).
USERRIGHTS	userId (FK) + rightCode (FK) as a composite PK	A ticked checkbox = one row. Unticking deletes the row. This is another junction table.

2.5 Following one sale through the tables

Sale **T-001** in Figure 25 is made of rows in three tables. Prices are looked up from PRICEHIST using the sale date, so an old sale keeps the price it was sold at even if the price changes later.

Table	Row(s)	Meaning
SALES	T-001 · 2026-10-01 · C-001 · E-002	Acme Corporation bought, sold by Juan Reyes
SALESDETAIL	T-001 · P-001 · 10 T-001 · P-002 · 2	Ten Industrial Widget A, two Steel Bracket Heavy
PRICEHIST (lookup)	P-001 → 13.75 · P-002 → 48.50	Price in force on the sale date
Calculated	10 × 13.75 = 137.50; 2 × 48.50 = 97.00	**Total = 234.50**, the figure shown on the Sales screen

```
-- Unit price of each line on sale T-001, as of the sale date (MySQL/PostgreSQL syntax)
SELECT d.prodCode, d.quantity,
       (SELECT p.unitPrice FROM PRICEHIST p
         WHERE p.prodCode = d.prodCode AND p.effDate <= s.salesDate AND p.status = 'Active'
         ORDER BY p.effDate DESC LIMIT 1) AS unitPrice
FROM SALESDETAIL d JOIN SALES s ON s.transNo = d.transNo
WHERE s.transNo = 'T-001' AND d.status = 'Active';
```

3 The screens, one module at a time

3.1 Getting in: landing, register, login, forgot password

The landing page is public. From there a visitor can register, or sign in. Two ideas to separate in your head: **authentication** (“who are you?” — the login) and **authorization** (“what may you do?” — roles and rights).

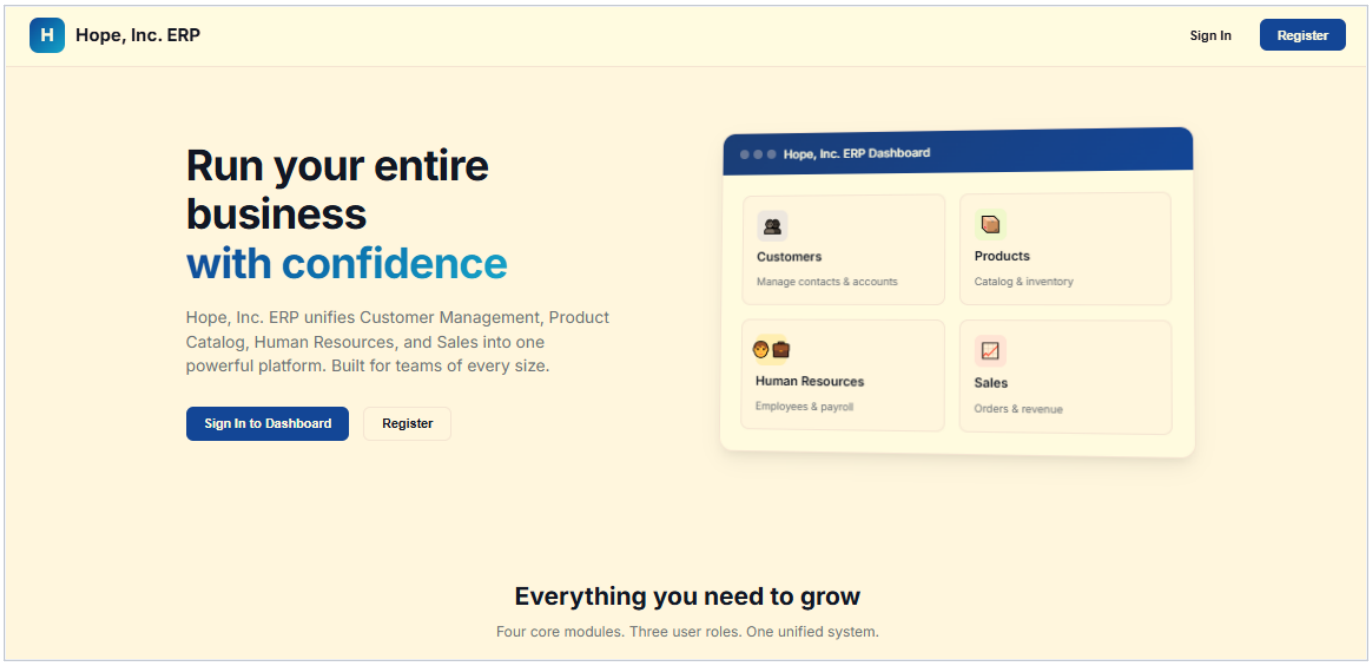


Figure 2. *Public landing page: it advertises the modules and offers Sign In and Register.*

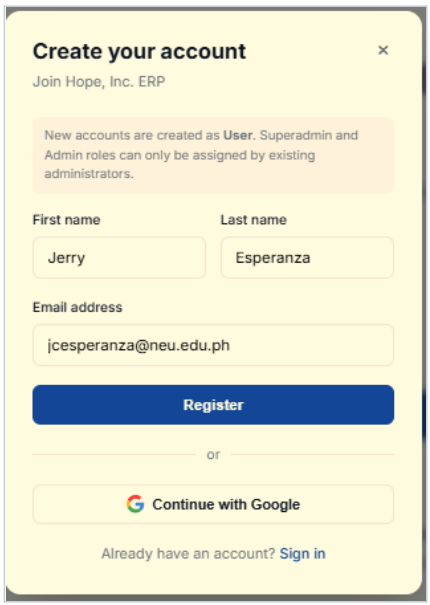


Figure 3. *Register. New accounts are always created as User; “Continue with Google” signs in through a third party.*

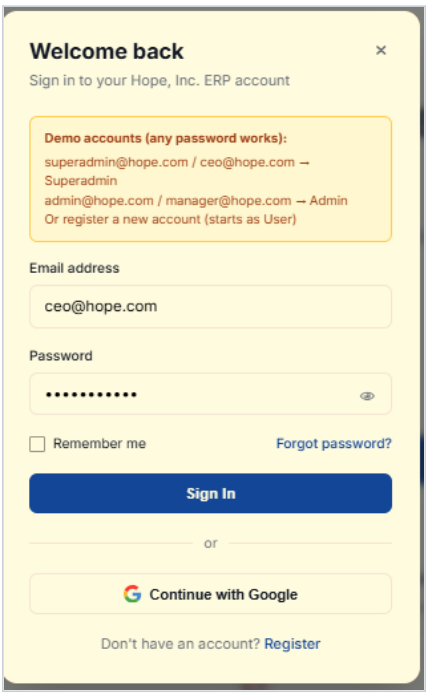


Figure 4. *Login. The demo accounts (any password works) exist only for the prototype.*

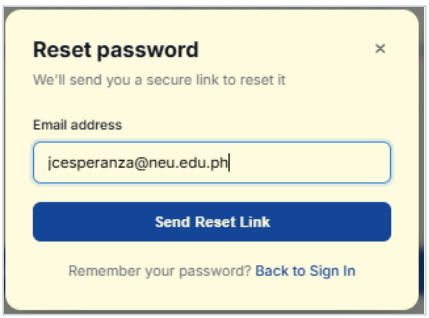


Figure 5. *Forgot password. A secure reset link is e-mailed.*

WATCH OUT

“Any password works” is acceptable for a classroom demo only. A real system compares a **salted hash** of the password (for example bcrypt or Argon2), locks or throttles repeated failures, and sends reset links that expire.

3.2 Dashboards: same system, different menus

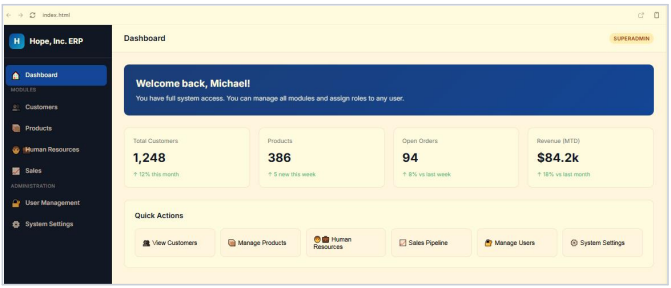


Figure 6. *Superadmin: full menu, including User Management and System Settings.*

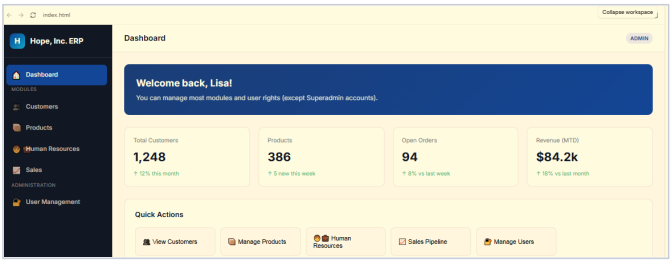


Figure 7. *Admin: all modules and User Management, but no Superadmin powers.*

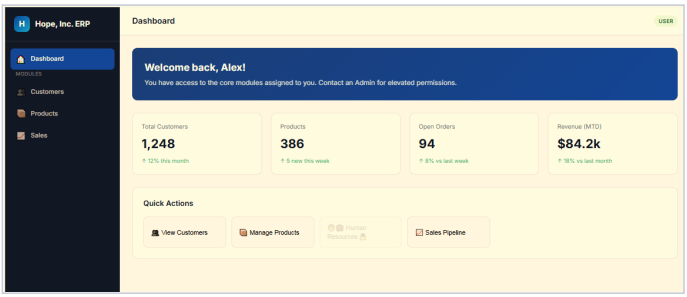


Figure 8. *User: only the modules granted to them. Human Resources is greyed out in Quick Actions.*

The menu follows the role and the rights. The server builds the sidebar and Quick Actions from what the logged-in person may use. The statistics on these cards (customers, products, open orders, revenue) are sample numbers in the prototype; in the real system each is a query such as `SELECT COUNT(*) FROM CUSTOMER WHERE status='Active'`.

3.3 Customers: your first CRUD screen

Start here when you build. CUSTOMER has no foreign keys, so it is the simplest table, and every other module repeats its pattern: list → search → add → edit → soft delete → restore. Numbers in the figure match the list below.

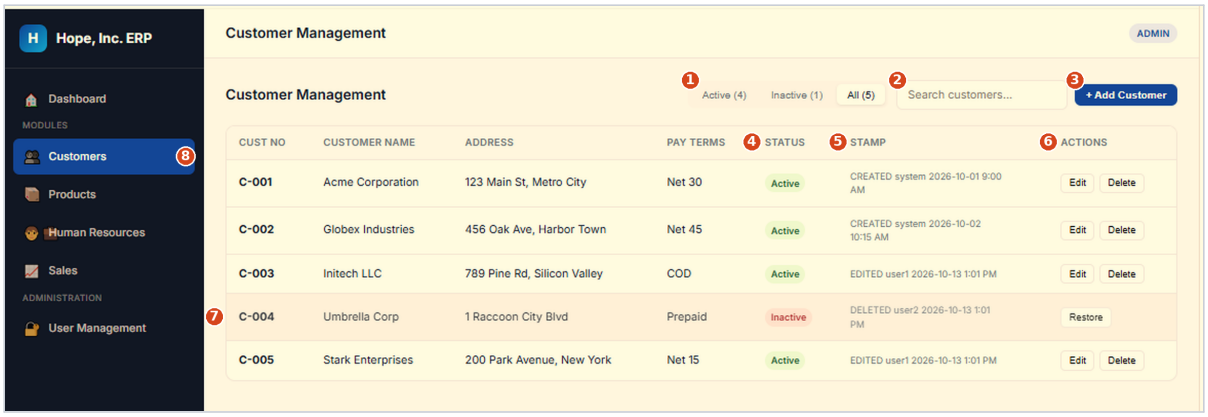


Figure 9. Customer Management, Admin view (all rows, including the soft-deleted one).

1	**Status tabs** — Active / Inactive / All with counts (Admin only)	5	**Stamp** — latest action, who did it, and when
2	**Search** — filters rows as you type	6	**Edit / Delete / Restore** — **Update** and **Delete**; Restore replaces them on a deleted row
3	**+ Add Customer** — **Create**; Status and Stamp are filled in by the system	7	**Shaded row** — a soft-deleted record that only Admins can see
4	**Status badge** — Active, or Inactive (soft-deleted)	8	**Sidebar** — the modules available to this role

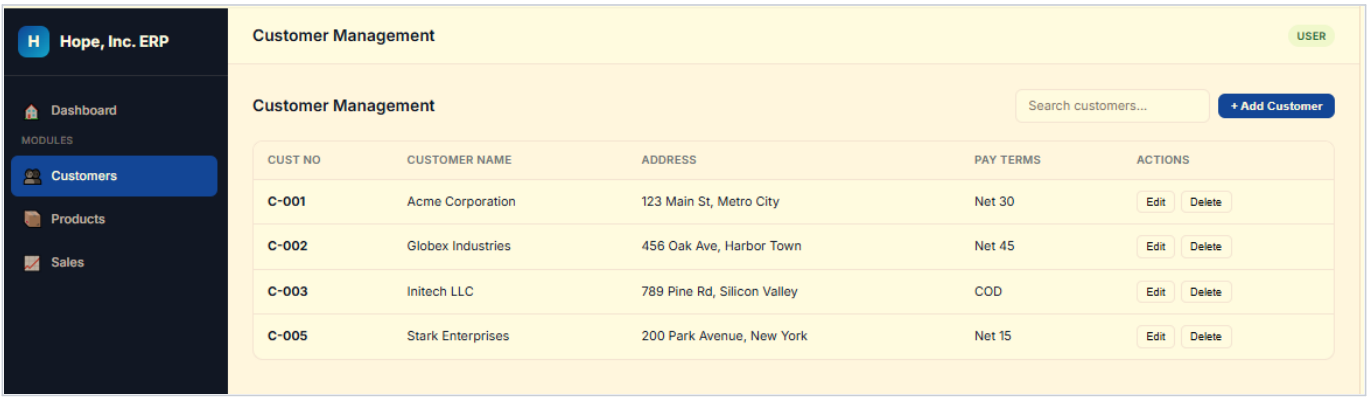


Figure 10. Customer Management, User view: no status tabs, no Status or Stamp columns, and C-004 is gone.

3.4 Products and their price history

A product's price changes over time, so it lives in its own table (PRICEHIST) with an **effective date**. The "Current Price" column shows the row with the latest effective date that is not in the future. P-004 shows "—" (no current price). The **Prices** button opens that product's price history.

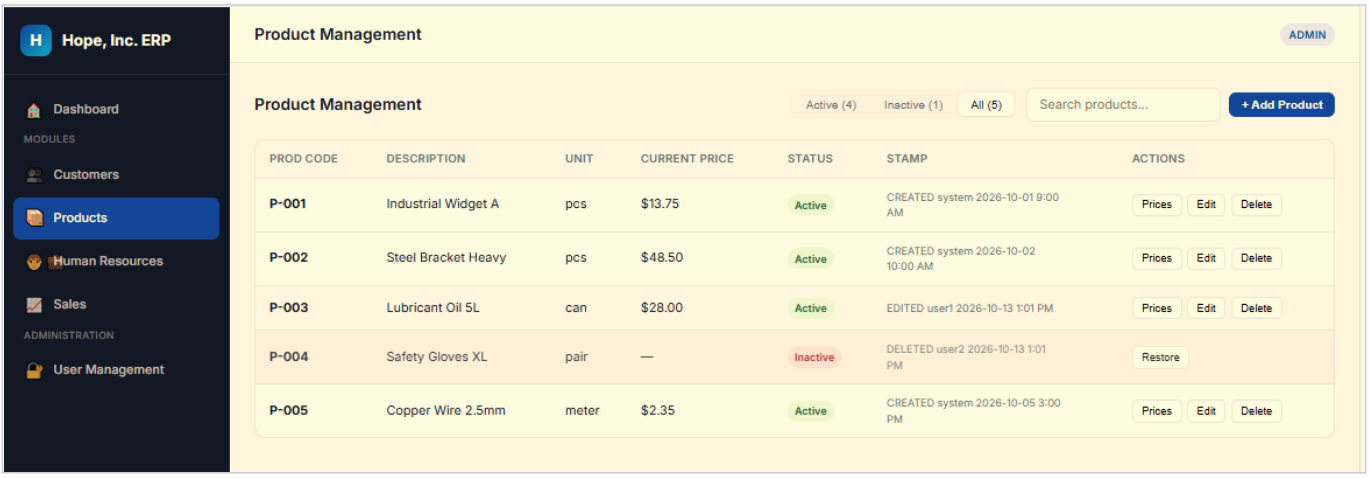


Figure 11. *Product Management, Admin view.*

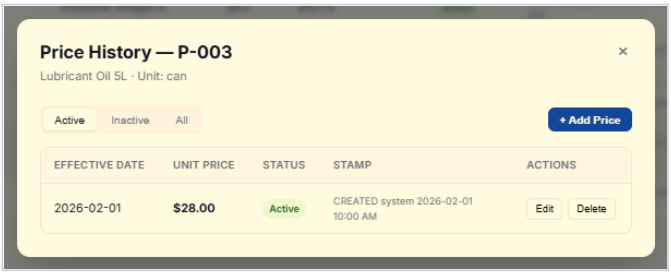


Figure 12. *Price History of P-003, Admin view (with Status and Stamp).*



Figure 13. *The same dialog, User view (Status and Stamp hidden).*

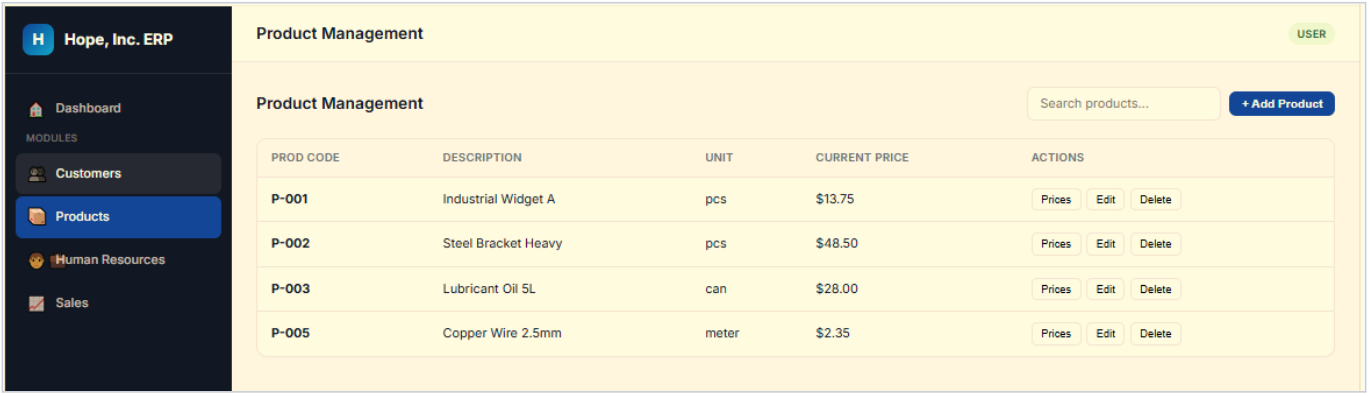


Figure 14. *Product Management, User view: only active products, no Status or Stamp.*

3.5 Human Resources: employees, jobs and departments

HR has three tabs. **Employees** holds the person; **Jobs** and **Departments** are small lookup tables. The **History** button on an employee opens **JOBHISTORY**: each promotion or raise is a new row, and the row with the latest effective date is the current job.

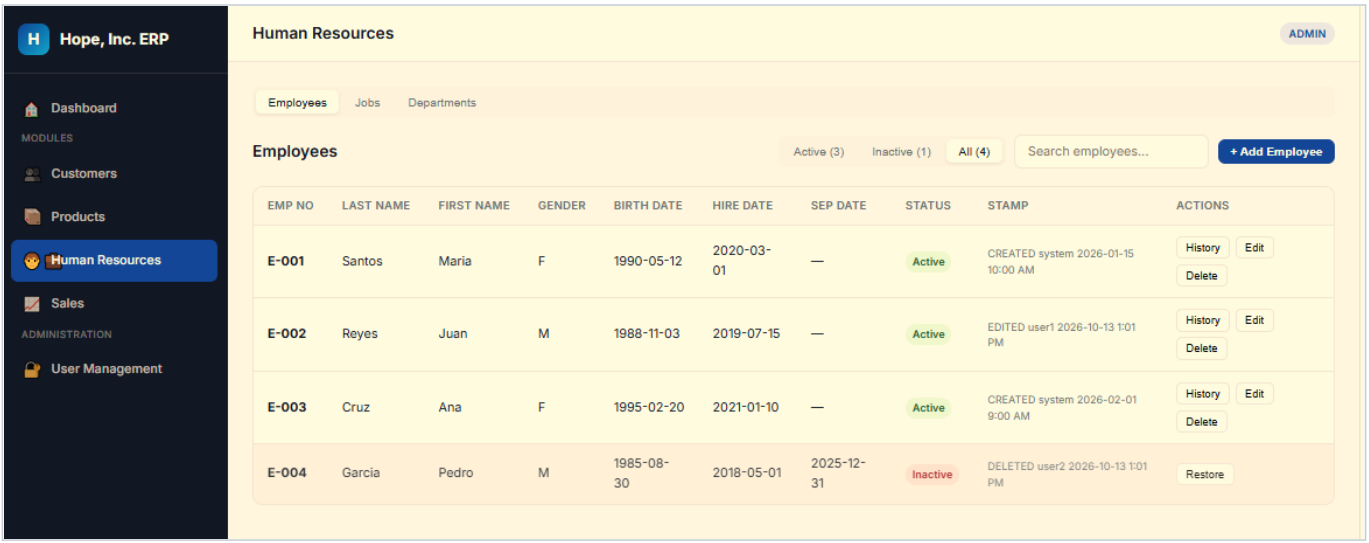


Figure 15. *Employees tab, Admin view. E-004 is deleted and also carries a separation date.*

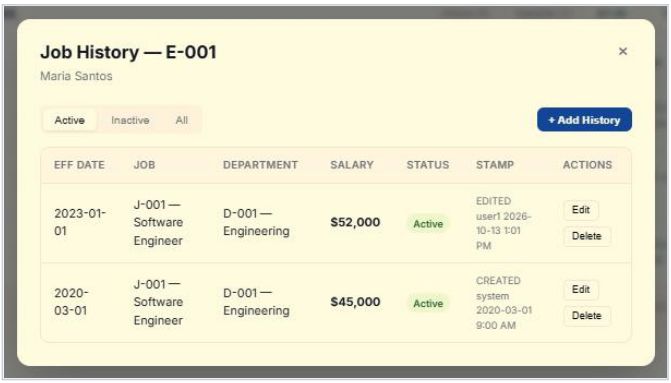


Figure 16. *Job History of E-001, Admin view.*



Figure 17. *Job History of E-001, User view.*



Figure 18. *Jobs tab, Admin view.*



Figure 19. *Departments tab, Admin view.*

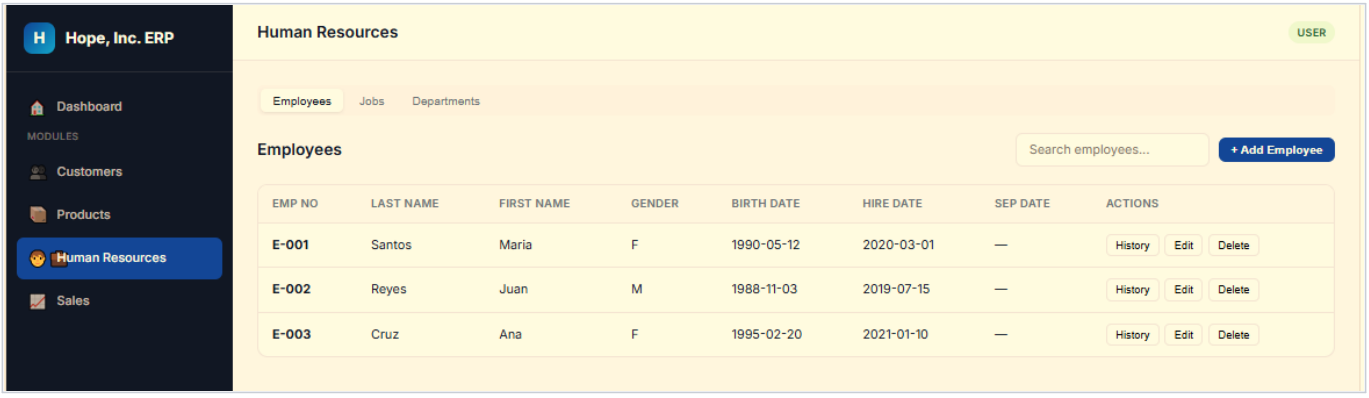


Figure 20. *Employees tab, User view.*



Figure 21. *Jobs tab, User view.*



Figure 22. *Departments tab, User view.*

WATCH OUT

Personal data. Birth dates and salaries are personal information. In the Philippines, the Data Privacy Act of 2012 (Republic Act No. 10173) expects you to collect only what you need and protect it from unauthorised access. In this system, that is the job of the HR rights in §3.8: decide who may *view* job history, not just who may edit it.

3.6 Sales: a header and its detail lines

A sale is stored as a **header** (SALES: one row) plus **detail lines** (SALESDETAIL: one row per product). This “master-detail” pattern appears in invoices, purchase orders and enrolment forms, so it is worth learning well. The list shows each sale’s **Total**, calculated from its lines. T-004 is deleted and shows $0.00.

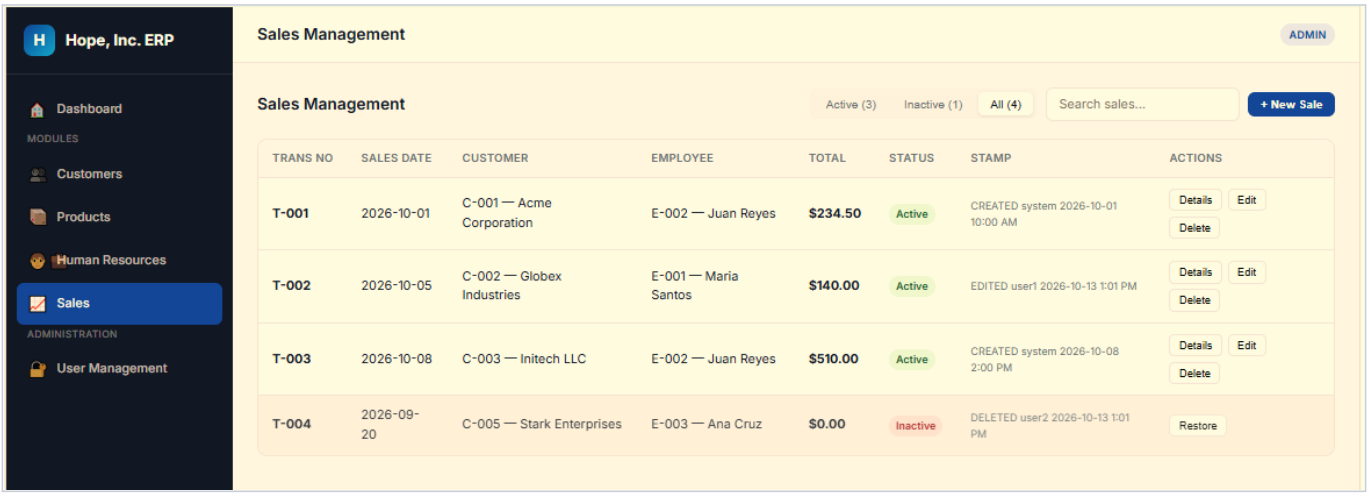


Figure 23. *Sales Management, Admin view.*

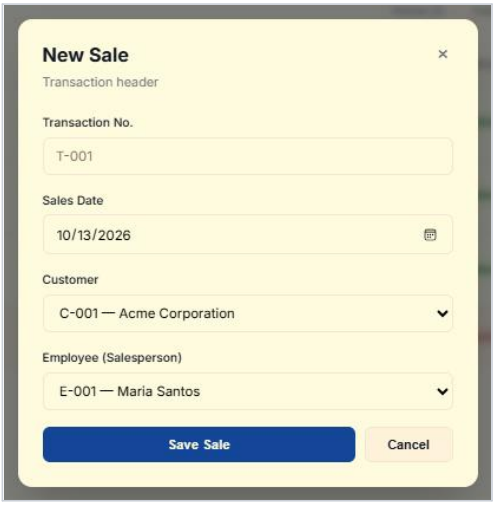


Figure 24. *New Sale form (header).*

Four rules of this form

- **Transaction No.** is auto-numbered and cannot be edited, so two sales can never share a number.
- **Customer** is a drop-down filled from the CUSTOMER table (a foreign key).
- **Employee** is a drop-down filled from the EMPLOYEE table (a foreign key).
- **Status and Stamp** are generated by the system, not typed.

Why drop-downs? You cannot type a customer that does not exist. That is **referential integrity** enforced in the screen as well as in the database.

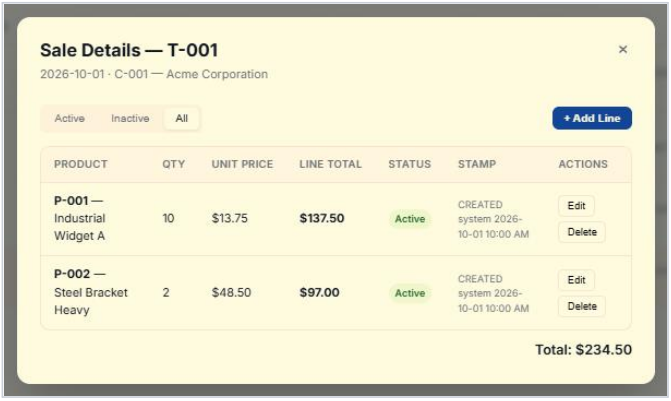


Figure 25. *Sale Details of T-001, Admin view: each line total is quantity × unit price.*

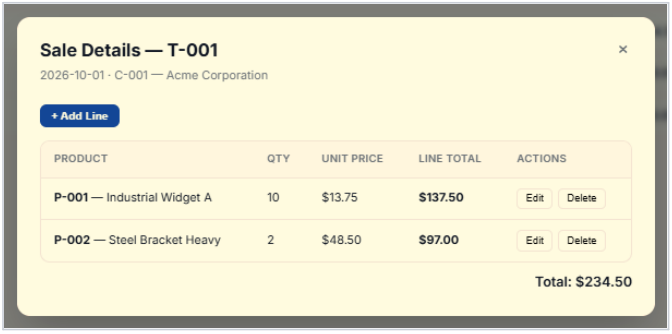


Figure 26. *Sale Details of T-001, User view.*



Figure 27. *Sales Management, User view.*

3.7 Reports

Reports only **read** data. Each one is a SQL query (usually with `JOIN`, `GROUP BY`, `SUM`, `ORDER BY` and `LIMIT` for "Top N") with **Print** and **Save as PDF** buttons. A User may open a report only after an Admin has granted that report's right.

Report	Question it answers	Inputs
Top Customers	Who buys the most?	From date, To date, Top N
Product Master List	What do we sell and at what current price?	—
Top Products	Which products earn the most (quantity and amount)?	From date, To date, Top N
Active Employees	Who currently works here? (sepDate is empty)	—
All Employees	Everyone, including those who have separated	—
Employee Job History	How did one employee's job and salary change?	Choose an employee
Top Sales	Which individual sales are the biggest?	From date, To date, Top N

```
-- Top N customers by sales between two dates.
-- v_saleline is a view: one row per active sale line with lineTotal = quantity x price on the sale date.
SELECT c.custNo, c.custName, SUM(l.lineTotal) AS totalSales
FROM v_saleline l JOIN CUSTOMER c ON c.custNo = l.custNo
WHERE l.salesDate BETWEEN :fromDate AND :toDate
GROUP BY c.custNo, c.custName
ORDER BY totalSales DESC
LIMIT :topN;                      -- SQL Server: SELECT TOP (:topN) ...
```

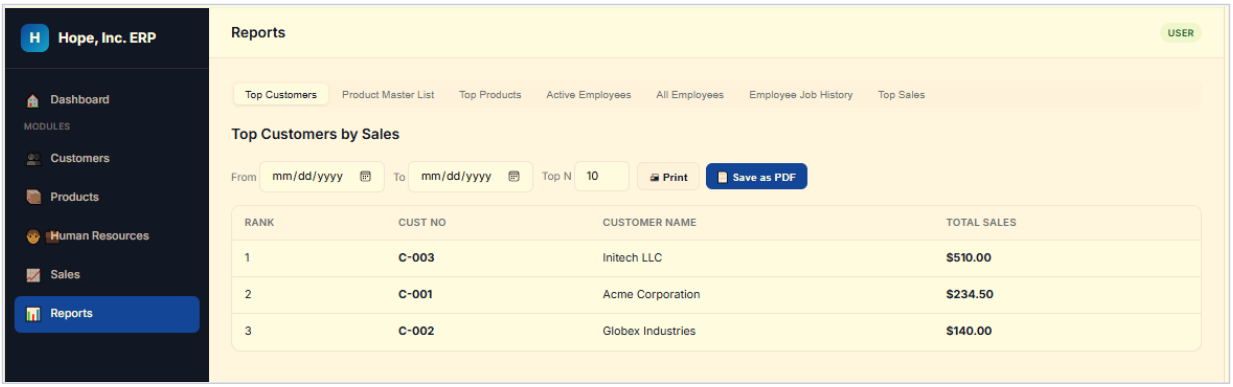


Figure 28. *Top Customers by Sales (shown from a User account that has been granted the report).*

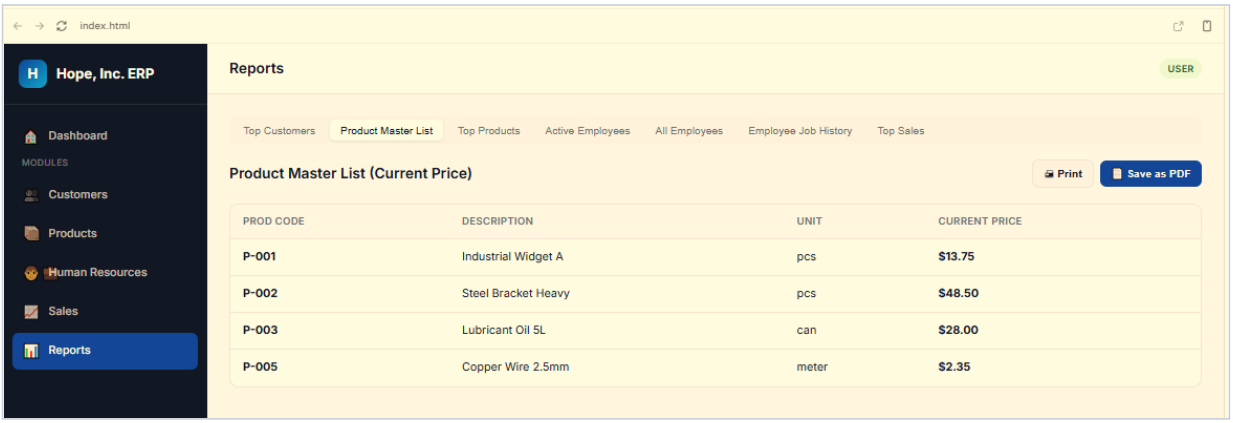


Figure 29. *Product Master List (Current Price). Only active products appear.*

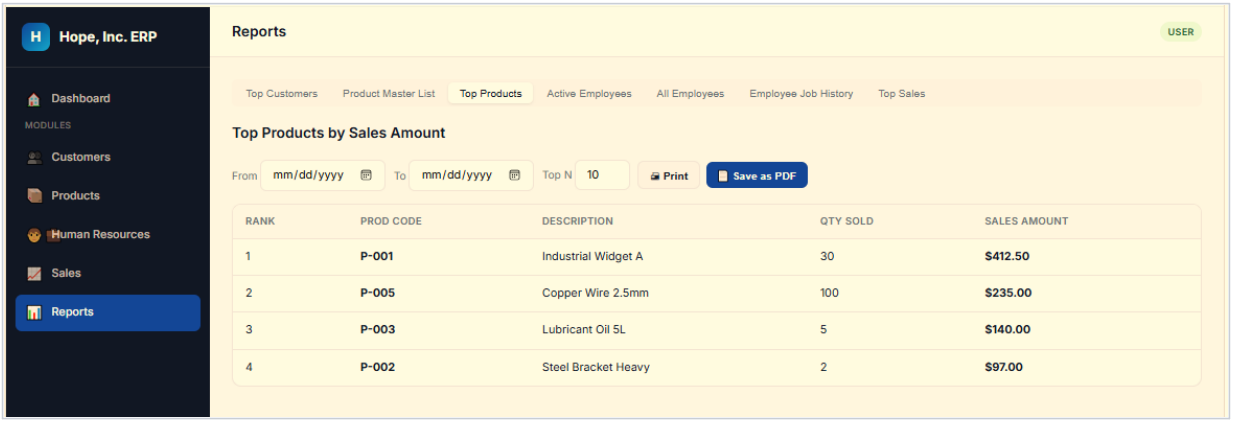


Figure 30. *Top Products by Sales Amount.*

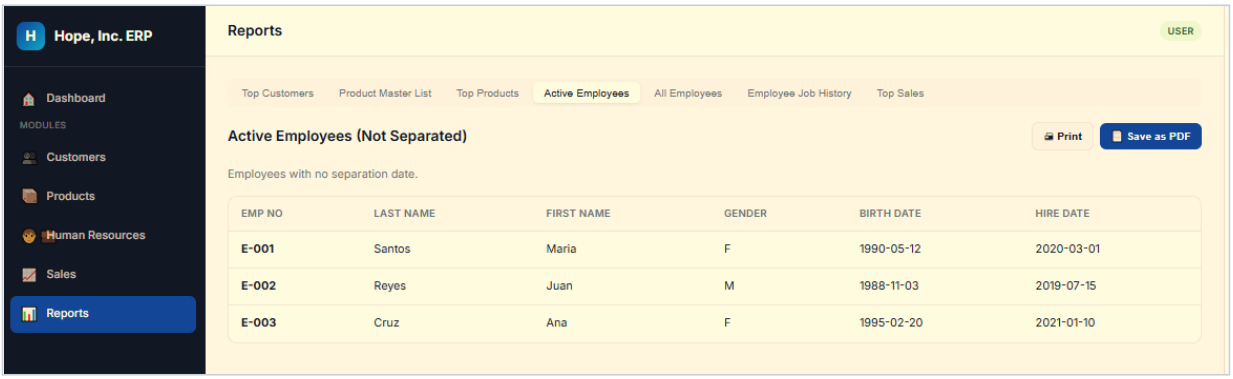


Figure 31. *Active Employees (Not Separated): employees with no separation date.*

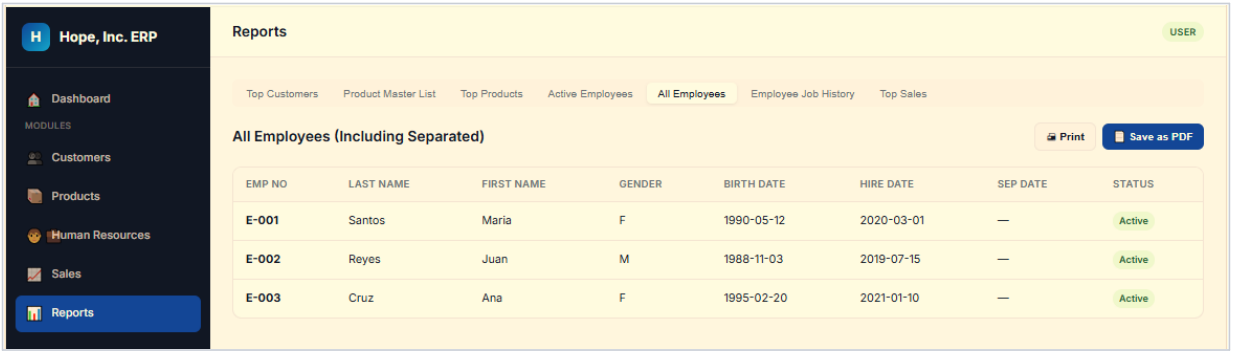


Figure 32. *All Employees (Including Separated).*

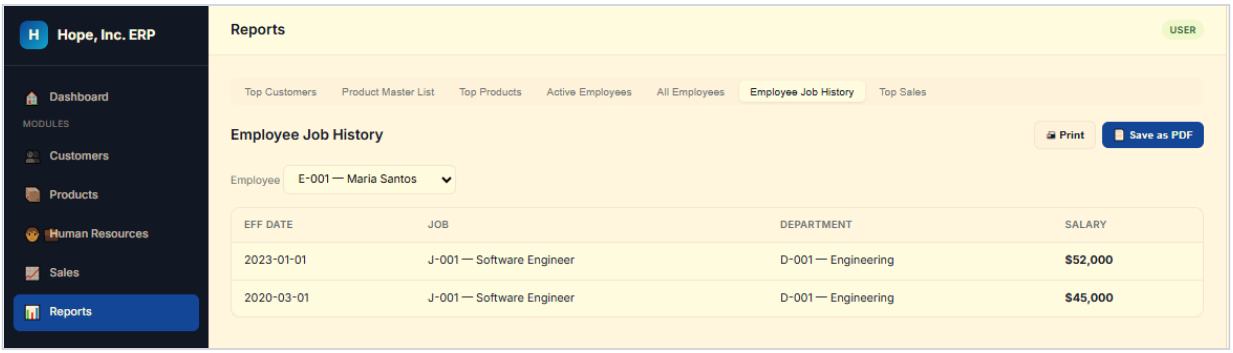


Figure 33. *Employee Job History for one chosen employee.*

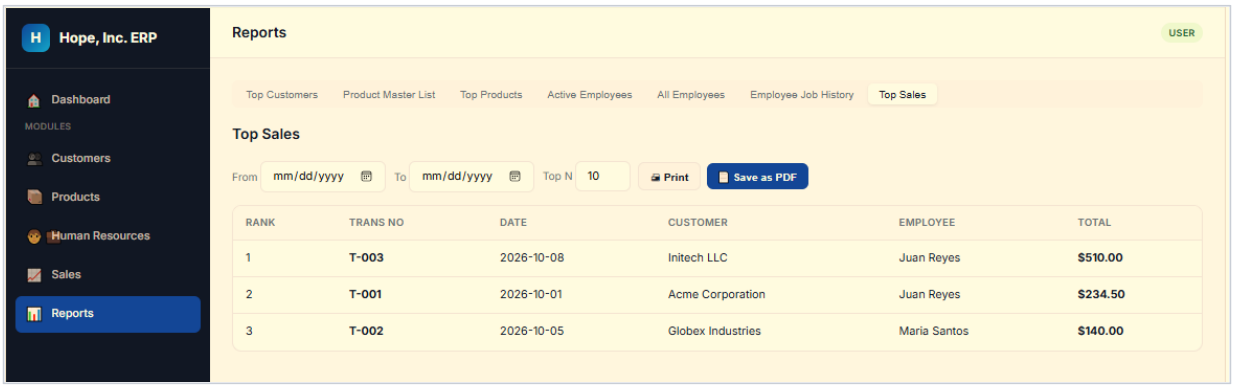


Figure 34. *Top Sales: the largest sales in a date range.*

TRY IT

Check the numbers. Add up the Top Products amounts ($412.50 + $235.00 + $140.00 + $97.00) and the Top Customers totals ($510.00 + $234.50 + $140.00). Both come to $884.50, which is what the three active sales add up to. When different reports agree like this, your queries are probably consistent.

3.8 User management: accounts, blocking and rights

Admins manage **Users** only: Superadmin accounts are hidden, and other Admin rows show "—" in Actions. For a User the actions are **Block / Unblock** (a blocked account cannot sign in), **Rights** and **Edit**. The Rights dialog is a set of checkboxes grouped by module. Ticking a module's box switches that module on; the boxes under it decide what the person may do there. **All** and **None** are shortcuts.

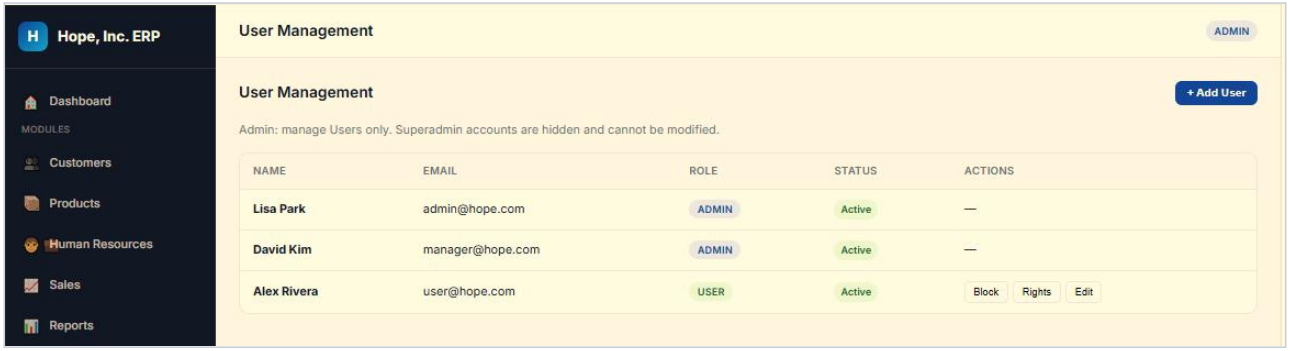


Figure 35. *User Management, Admin view. Alex Rivera is an active User.*

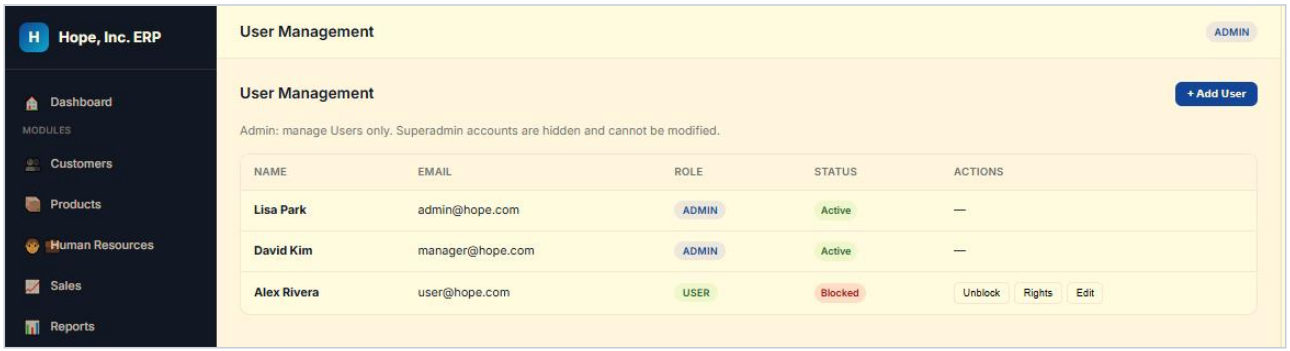


Figure 36. The same user after being blocked: the status turns red and the button becomes Unblock.

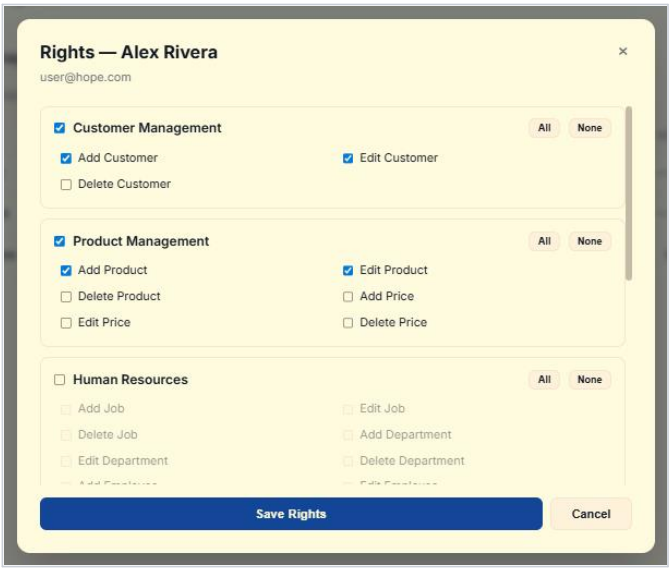


Figure 37. Rights: Customer and Product Management.

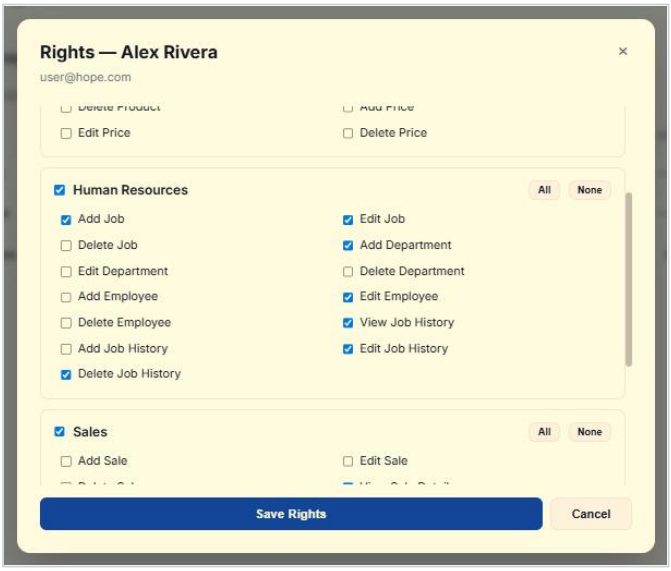


Figure 38. Rights: Human Resources.

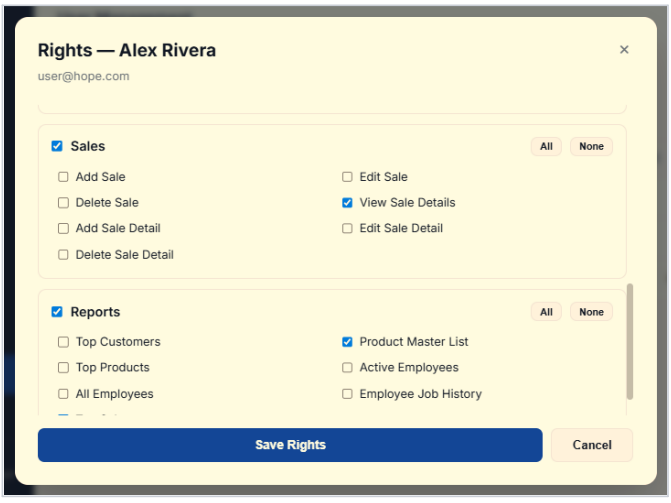


Figure 39. Rights: Sales Management.

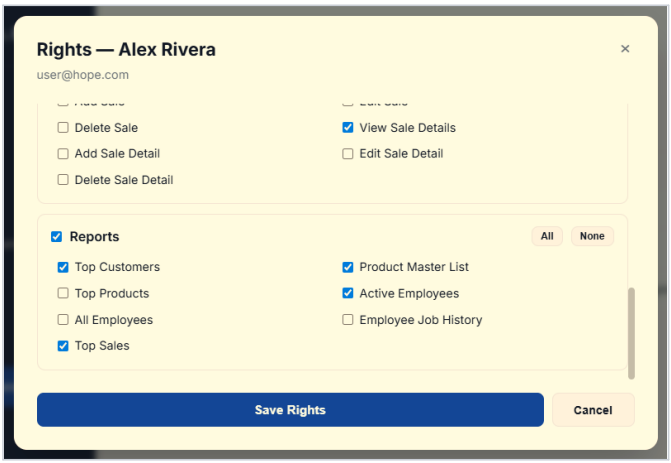


Figure 40. Rights: Reports.

Module	Rights in the dialog	Count
Customer	Add, Edit, Delete Customer	3
Product	Add, Edit, Delete Product; Add, Edit, Delete Price	6
Human Resources	Add, Edit, Delete for Job, Department, Employee; View, Add, Edit, Delete Job History	13
Sales	Add, Edit, Delete Sale; View, Add, Edit, Delete Sale Detail	7

Module	Rights in the dialog	Count
Reports	Top Customers, Product Master List, Top Products, Active Employees, All Employees, Employee Job History, Top Sales	7

TRY IT

Spot the gap. In the sample screenshots, Alex Rivera's rights (Figures 37–40) leave *Delete Customer*, *Delete Product* and the price rights unticked, yet the User views (Figures 10, 14, 13) still show Delete and Prices buttons. The screens may have been captured at different times, but ask yourself: **should the button be hidden, and where must the check happen so that it cannot be bypassed?** (Hint: hiding a button is a courtesy; the server must refuse the action.)

4 Building it

4.1 Suggested build order

Step	Build	Why in this order
1	Create the database: tables, keys, and the status and stamp columns	Everything else depends on it
2	Insert sample data (the rows in the screenshots make a good set)	Lets you test screens immediately
3	Register, login and roles	Every later screen needs to know who is logged in
4	Customers: full CRUD, soft delete, restore, stamp	Simplest table; the template for all other modules
5	Products and price history	Your first parent-child (one-to-many) screen
6	Human Resources: Jobs, Departments, Employees, then Job History	Lookup tables first, then the tables that reference them
7	Sales header and detail lines	Needs customers, employees and priced products to exist
8	Reports	They read data from every module, so build them last
9	User Management and rights	Wire the rights into every earlier screen and test them

4.2 From prototype to real system

The prototype shows …	A real system must …
Any password is accepted for demo accounts	Store salted password hashes and verify them; limit failed attempts
Buttons hidden for some roles	Also check the user's role and rights on the server for every request
Dashboard numbers typed in by hand	Calculate them with queries (COUNT, SUM) from the live tables
Users do not see inactive rows	Filter in SQL (`WHERE status = 'Active'`) on the server, not just in the page
PAYMENT is in the ERD but has no screen	Add a Payments screen (ORNo, date, amount, linked to a sale) so you can show balance = total − payments
Free typing in forms	Validate input (required fields, dates, a separation date never before the hire date)

4.3 Test every role

Sign in as …	Check that …
Superadmin / Admin	Deleted rows are visible and shaded; Status and Stamp columns appear; Restore works; the stamp changes after each Add, Edit, Delete

Sign in as …	Check that …
User	No deleted rows, no Status or Stamp; only the modules and buttons the rights allow; blocked users cannot sign in
Anyone	Totals equal the sum of their lines; reports agree with each other; a sale cannot reference a missing customer, employee or product

4.4 Key terms

Term	Meaning
Entity / table	A thing the system keeps data about (customer, product, sale …)
Primary key (PK)	Column(s) that identify a row uniquely; composite when more than one column
Foreign key (FK)	A column holding another table's PK; it links rows and enforces integrity
Junction table	A table (e.g. SALESDETAIL) that turns a many-to-many link into two one-to-many links
CRUD	Create, Read, Update, Delete: the four basic operations on data
Soft delete	Marking a row Inactive instead of physically removing it
Stamp	A record of the latest action on a row, who did it and when
Authentication	Proving who you are (login)
Authorization	Deciding what you may do (roles and rights)
Derived value	Data calculated from other data (totals, current price) rather than stored